

QK Relational Architecture

Minimal Runnable PyTorch Implementation

2 layers - $d = 48$ - headless A_0 - separate strict A_h - 80,352 parameters

Publication date: 14 September 2026

Purpose

This document contains one self-contained Python program sufficient to instantiate, smoke-test and train the tested small QK language architecture. It implements the same current mathematics used in the competitor audit: distance-aware ordinary reading, separate final-block hop projections, a strict no-self hop mask, and explicit $A_0 A_h^k H$ propagation.

Requirements:

```
python -m pip install torch numpy
```

Smoke test:

```
python qk_minimal.py --smoke
```

Expected key line: parameters: 80352.

Minimal byte-level training:

```
python qk_minimal.py --train-byte-file version9.txt \  
--steps 4096 --context 128 --batch 8 --seed 11
```

Normal language training uses `extra_hops=0`. Requested hops are an explicit forward argument and are applied only in the final block.

Accuracy checks performed on this listing

Before this document was generated, the exact code below was executed successfully. Its smoke test passed the parameter-count, mask, finite-output and hop-gradient checks. The code was also loaded with the same state dictionary as the saved competitor-audit implementation; the two implementations produced bit-identical logits on the same random inputs for 0, 1, 3 and 8 requested hops. A two-step byte-training run completed with finite losses and saved weights. This verifies that the listing below is executable and implements the tested equations exactly.

Complete code: `qk_minimal.py`

```
1 """Minimal runnable PyTorch implementation of the tested QK architecture.  
2  
3 Architecture used in the September 2026 competitor audit:  
4 - 2 layers, d_model=48, byte vocabulary=256  
5 - no attention heads and no learned value projection in QK blocks
```

```

6 - ordinary field A0: separate Q0/K0 + fixed distance prior + causal self-allowed mask
7 - hop field Ah (final block only): separate Qh/Kh + strict causal no-self mask
8 - explicit extra hops: A0 @ Ah^k @ H, implemented by recurrent Ah @ state
9 - FF widths: 240 in block 1, 192 in block 2
10 - total parameters: 80,352
11
12 This file is intentionally compact and self-contained. It can train as a byte-level
13 next-token language model on any local file and can also run a smoke test without data.
14
15
16 import argparse
17 import math
18 from pathlib import Path
19
20 import numpy as np
21 import torch
22 from torch import nn
23 from torch.nn import functional as F
24
25
26 def distance_prior_log(n: int, device, dtype):
27     """Return delta=i-j and log D(|i-j|), with D(r)=.02/(1+r)+.98/(1+r)^2."""
28     pos = torch.arange(n, device=device)
29     delta = pos[:, None] - pos[None, :]
30     r = delta.abs().to(dtype)
31     log_prior = torch.log(0.02 / (1.0 + r) + 0.98 / (1.0 + r).square())
32     return delta, log_prior
33
34
35 def qk_fields(h, q0, k0, qh=None, kh=None, need_hop=False):
36     """Build ordinary A0 and, optionally, strict hop Ah.
37
38     Shapes:
39         h, q0, k0, qh, kh: [batch, sequence, d_model]
40         A0, Ah:             [batch, sequence, sequence]
41
42     """
43     n = h.shape[1]
44     d = q0.shape[-1]
45     delta, log_prior = distance_prior_log(n, h.device, h.dtype)
46
47     # Ordinary causal reading: j <= i, so the diagonal/self is legal.
48     ordinary_mask = delta >= 0
49     scores0 = (q0 @ k0.transpose(-2, -1)) / math.sqrt(d)
50     scores0 = scores0 + log_prior
51     A0 = scores0.masked_fill(~ordinary_mask, -torch.inf).softmax(dim=-1)
52
53     Ah = None
54     if need_hop:
55         if qh is None or kh is None:
56             raise ValueError("qh and kh are required when need_hop=True")
57
58     # Explicit hop: j < i. A hop must move to an earlier relation.
59     strict_mask = delta > 0

```

```

59     strict_mask = strict_mask.clone()
60
61     # Row 0 has no earlier key. A single fallback self entry keeps softmax finite.
62     # No later row is allowed to self-loop.
63     strict_mask[0, 0] = True
64
65     scoresh = (qh @ kh.transpose(-2, -1)) / math.sqrt(d)
66     Ah = scoresh.masked_fill(~strict_mask, -torch.inf).softmax(dim=-1)
67
68     return A0, Ah
69
70
71 class QKBlock(nn.Module):
72     def __init__(self, d_model: int, ff_dim: int, has_hop: bool):
73         super().__init__()
74         self.has_hop = has_hop
75         self.ln1 = nn.LayerNorm(d_model)
76         self.ln2 = nn.LayerNorm(d_model)
77
78         self.q0 = nn.Linear(d_model, d_model, bias=False)
79         self.k0 = nn.Linear(d_model, d_model, bias=False)
80
81         if has_hop:
82             self.qh = nn.Linear(d_model, d_model, bias=False)
83             self.kh = nn.Linear(d_model, d_model, bias=False)
84
85         self.ff = nn.Sequential(
86             nn.Linear(d_model, ff_dim, bias=False),
87             nn.ReLU(),
88             nn.Linear(ff_dim, d_model, bias=False),
89         )
90
91     def forward(self, x, extra_hops: int = 0):
92         h = self.ln1(x)
93         q0, k0 = self.q0(h), self.k0(h)
94
95         if extra_hops > 0:
96             if not self.has_hop:
97                 raise ValueError("extra_hops may only be applied to the hop-enabled
98 final block")
99             qh, kh = self.qh(h), self.kh(h)
100             A0, Ah = qk_fields(h, q0, k0, qh, kh, need_hop=True)
101
102             # A0 Ah ~k H: apply Ah repeatedly to H, then make the ordinary A0 read.
103             z = h
104             for _ in range(extra_hops):
105                 z = Ah @ z
106             out = A0 @ z
107         else:
108             A0, _ = qk_fields(h, q0, k0, need_hop=False)
109             out = A0 @ h
110
111     x = x + out
112     return x + self.ff(self.ln2(x))

```

```

112
113
114 class QKLanguageModel(nn.Module):
115     def __init__(self, vocab_size=256, d_model=48, max_context=2048):
116         super().__init__()
117         self.vocab_size = vocab_size
118         self.d_model = d_model
119
120         self.embed = nn.Embedding(vocab_size, d_model)
121         self.blocks = nn.ModuleList([
122             QKBlock(d_model, ff_dim=240, has_hop=False),
123             QKBlock(d_model, ff_dim=192, has_hop=True),
124         ])
125         self.norm = nn.LayerNorm(d_model)
126         self.head = nn.Linear(d_model, vocab_size, bias=False)
127
128         # Fixed sinusoidal positions, matching the tested small language model.
129         pos = torch.arange(max_context)[: , None].float()
130         div = torch.exp(
131             torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model)
132         )
133         pe = torch.zeros(max_context, d_model)
134         pe[:, 0::2] = torch.sin(pos * div)
135         pe[:, 1::2] = torch.cos(pos * div)
136         self.register_buffer("position", pe, persistent=False)
137
138         self.apply(self._init_weights)
139
140     @staticmethod
141     def _init_weights(module):
142         if isinstance(module, (nn.Linear, nn.Embedding)):
143             nn.init.normal_(module.weight, std=0.02)
144
145     def forward(self, tokens, extra_hops: int = 0):
146         if tokens.shape[1] > self.position.shape[0]:
147             raise ValueError("sequence is longer than max_context")
148
149         h = self.embed(tokens) * math.sqrt(self.d_model)
150         h = h + self.position[: tokens.shape[1]]
151
152         h = self.blocks[0](h, extra_hops=0)
153         h = self.blocks[1](h, extra_hops=extra_hops)
154         return self.head(self.norm(h))
155
156
157 def parameter_count(model):
158     return sum(p.numel() for p in model.parameters())
159
160
161 def make_byte_batch(data, batch_size, context, rng):
162     starts = rng.integers(0, len(data) - context - 1, size=batch_size)
163     windows = torch.stack([
164         data[int(s): int(s) + context + 1] for s in starts
165     ])

```

```

166     return windows[:, :-1], windows[:, 1:]
167
168
169 def train_byte_file(path, steps=1000, context=128, batch_size=8, seed=11, device="cpu"
170 ):
171     """Small next-byte training loop. Normal language training uses extra_hops=0."""
172     torch.manual_seed(seed)
173     np_rng = np.random.default_rng(seed + 10000)
174
175     raw = Path(path).read_bytes()
176     if len(raw) < context + 2:
177         raise ValueError("training file is too small for the selected context")
178
179     data = torch.from_numpy(np.frombuffer(raw, dtype=np.uint8).astype(np.int64))
180     model = QKLanguageModel().to(device)
181     optimizer = torch.optim.AdamW(model.parameters(), lr=5e-4, weight_decay=0.01)
182
183     model.train()
184     for step in range(1, steps + 1):
185         x, y = make_byte_batch(data, batch_size, context, np_rng)
186         x, y = x.to(device), y.to(device)
187
188         optimizer.zero_grad(set_to_none=True)
189         logits = model(x, extra_hops=0)
190         loss = F.cross_entropy(logits.reshape(-1, 256), y.reshape(-1))
191         loss.backward()
192         nn.utils.clip_grad_norm_(model.parameters(), 1.0, error_if_nonfinite=True)
193         optimizer.step()
194
195         if step == 1 or step % 100 == 0 or step == steps:
196             print(f"step={step:5d} loss={loss.item():.4f}")
197
198     return model
199
200 def smoke_test():
201     torch.manual_seed(123)
202     model = QKLanguageModel()
203     assert parameter_count(model) == 80352, parameter_count(model)
204
205     x = torch.randint(0, 256, (2, 32))
206     y0 = model(x, extra_hops=0)
207     y3 = model(x, extra_hops=3)
208     assert y0.shape == (2, 32, 256)
209     assert y3.shape == (2, 32, 256)
210     assert torch.isfinite(y0).all() and torch.isfinite(y3).all()
211
212     # Mask sanity check.
213     h = model.blocks[1].ln1(
214         model.embed(x) * math.sqrt(model.d_model) + model.position[: x.shape[1]]
215     )
216     b = model.blocks[1]
217     A0, Ah = qk_fields(h, b.q0(h), b.k0(h), b.qh(h), b.kh(h), need_hop=True)
218     assert torch.allclose(A0.sum(-1), torch.ones_like(A0.sum(-1)), atol=1e-6)

```

```

219 assert torch.allclose(Ah.sum(-1), torch.ones_like(Ah.sum(-1)), atol=1e-6)
220 diag = torch.diagonal(Ah, dim1=-2, dim2=-1)
221 assert torch.all(diag[:, 1:] == 0), "strict hop rows must not self-loop"
222
223 # qh/kh are dormant at k=0 and active when hops are requested.
224 model.zero_grad(set_to_none=True)
225 model(x, extra_hops=0).sum().backward()
226 assert b.qh.weight.grad is None and b.kh.weight.grad is None
227
228 model.zero_grad(set_to_none=True)
229 model(x, extra_hops=1).sum().backward()
230 assert b.qh.weight.grad is not None and torch.isfinite(b.qh.weight.grad).all()
231 assert b.kh.weight.grad is not None and torch.isfinite(b.kh.weight.grad).all()
232
233 print("smoke test: PASS")
234 print("parameters:", parameter_count(model))
235 print("normal logits:", tuple(y0.shape))
236 print("+3-hop logits:", tuple(y3.shape))
237
238
239 def main():
240     parser = argparse.ArgumentParser()
241     parser.add_argument("--smoke", action="store_true", help="run architecture checks"
242 )
243     parser.add_argument("--train-byte-file", type=str, default=None,
244                         help="train a next-byte model on a local file")
245     parser.add_argument("--steps", type=int, default=1000)
246     parser.add_argument("--context", type=int, default=128)
247     parser.add_argument("--batch", type=int, default=8)
248     parser.add_argument("--seed", type=int, default=11)
249     parser.add_argument("--device", type=str, default="cpu")
250     args = parser.parse_args()
251
252     if args.smoke or args.train_byte_file is None:
253         smoke_test()
254
255     if args.train_byte_file:
256         model = train_byte_file(
257             args.train_byte_file,
258             steps=args.steps,
259             context=args.context,
260             batch_size=args.batch,
261             seed=args.seed,
262             device=args.device,
263         )
264         torch.save(model.state_dict(), "qk_language_model.pt")
265         print("saved qk_language_model.pt")
266
267 if __name__ == "__main__":
268     main()

```

Licensing

Documentation and explanatory presentation are licensed under Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0). Source code and code listings implementing the experimental architecture are licensed separately under the PolyForm Noncommercial License 1.0.0. Neither licence grants unrestricted commercial-use rights.